

Primary Objective:

- **Use Deep Q-Network for exact solver.**
- **Fit exact solver, then run with DQN**
- **Start with $Q = 0$, or weight = 0. Then, see if DQN can converge.**
- **Can be arbitrarily large, but DO NOT USE Convolutional neural networks.**
- **Ask for Q Matrix of every scenario (Minimax, always I**
- **left, random, and best response scenarios)**

1. Using Deep Q-Network for exact solver

- Training: minimax against minimax
- Take the solution and test the win fraction against the four opponents
- EXTRACT THE EXACT OPTIMAL POLICY
- Show Q-matrices for interesting states
 - Mixed strategy states, opponent right in front, diagonal, distance > 1 , let arrow thickness represent probability (U:50 $\rightarrow$ thick arrow, L:25 $\rightarrow$ thin)

Remember: P1 wants to MINIMIZE P0's value, so in the Q matrices P1 likes the column with the lowest values.

The negative values in the Q-matrix represent the scenario where P0 makes a mistake and P1 makes the correct response TO THAT MISTAKE.

The positive values in the Q-matrix represent the opposite: P1 makes a mistake, and P0 makes the correct response TO THAT MISTAKE.

1. Chart #1: P0 will want to move up, and P1 will try to cut it off by moving right
2. Chart #2: P0 up, P1 goes down to prevent P0 from going to the right and escaping
3. Chart #3: P0 up, P1 up
4. Chart #4: At the start state both players will want to move up
5. Chart #5: P0 up, P1 to the right
6. Chart #6: P0 left, P1 up

Win Fractions

Opponent	Win %	W / T / L
Minimax	32.6%	775 / 830 / 775
Always-Left	65.6%	1561 / 0 / 819
Random	78.9%	1878 / 114 / 388
Best-Response	32.6%	775 / 830 / 775